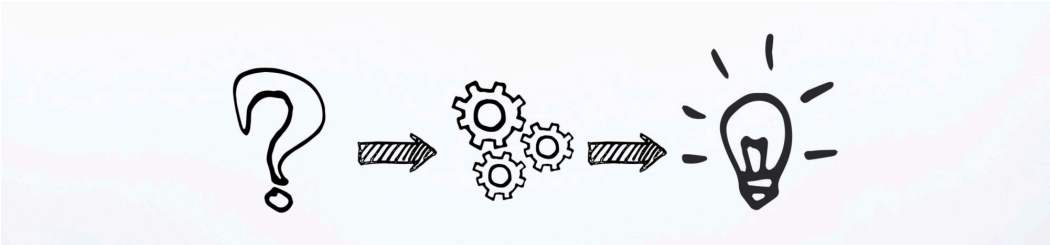


Problem Solving

Problem Solving - A Recent Use Case

Kunal Patil
Gen AI Solution Architect | Agentic AI • RAG • LangChain • AWS Bedrock | Cloud-Native BFSI & Healthcare Leader | AW...

July 5, 2022

> "If I had an hour to solve a problem I'd spend 55 minutes thinking about the problem and 5 minutes thinking about solutions."

> - Albert Einstein

Recently I came across an interesting problem statement which required a bit of reverse engineering with on-premise and cloud solution. Here is the problem solving journey.

Problem Statement

The organization's business was severely impacted due to frequent downtime of middleware APIs. The goal was to improve overall uptime and performance of middleware APIs. We were dealing with business critical applications having 600+ APIs, 120+ internal/external consumers and 5 business units getting impacted.

The Plan

After initial discussions with stakeholders, following was the high level plan:

Phase 1 (short term goal):

- Root cause analysis for frequent downtime
- Propose corrective actions to improve uptime

Phase 2 (long term goal):

- Analyze the existing architecture
- Propose blueprint for improved architecture

I will be detailing only the Phase 1 in this article; for phase 2, I may publish another article.

Root Cause Analysis

After multiple discussions with stakeholders, support team, development team and analyzing application logs and configurations following were the findings:

- No rate limits defined for API
- Inefficient API timeout configurations
- Lack of API caching

The above mentioned factors were overburdening API and ultimately downstream systems which was causing the downtime.

Proposed Corrective Actions

Considering root cause analysis, it was identified that we need to define API rate limits, efficient timeout values and enable API caching with appropriate time-to-live (TTL) values.

Few other hurdles were that, analytics reports or usage data was not available to identify reasonable values for parameters mentioned to improve the uptime. Fortunately there were access logs at API level which could be used to identify analytics data and usage patterns. Few examples of data points in access logs - Request/Response timestamps, consumer ID, Error codes, Timeout events, etc.

The Solution

I decided to follow reverse engineering to generate required analytics data and usage patterns based on the access logs to eventually come up to reasonable values for API - rate limit, timeout, and cache TTL.

Few more factors to be considered for the solution:

- Logs doesn't have consistency across and within API e.g. for timestamp fields used were "Request_Time", "Request-in time", "Req.-timestamp", etc.
- Logs contained sensitive data which should not be processed/stored as per organization policy e.g. Customer ID/Name
- Existing application was on-premises and any new solution should be implemented on cloud as per organization strategy

High Level Workflow:

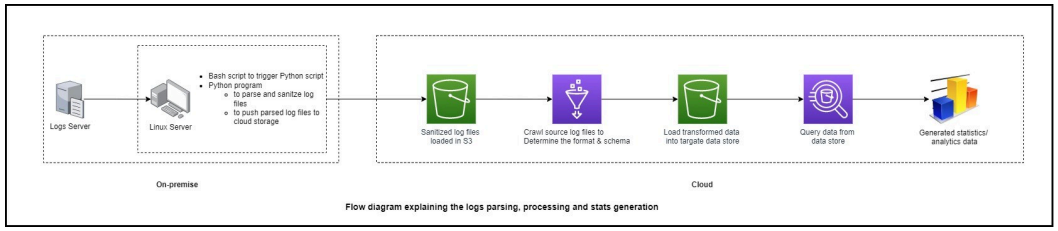
- Process the log files (for last 3 years) on-premise for data sanitization, masking sensitive data and to have a common format with consistent data points
- Push the sanitized log files to cloud storage
- Process sanitized log files to generate analytics reports and usage patterns which should include data points (per API per consumer) like Average API call, Average timeout occurrences, Average Response time, Error patterns, Peak API utilization, API utilization pattern across the day/month
- Use the analytics data and usage pattern to define and implement API Rate Limit, Timeout and cache TTL
- Monitor the API to track resulting uptime

Tech Stack:

- Linux bash script - To trigger log file sanitization
- Python - To parse and sanitize log files and push it to cloud storage

- AWS - S3 (file storage), Glue (Automatic schema discovery with crawler), Athena (Query data to generate analytics reports and usage patterns)

Following diagram would depict solution:



Results

The solution was implemented successfully and after couple of months of monitoring it's observed that API uptime has **improved by 78%** which is quite impressive. Further I am planning to integrate AWS ML tools with Athena to generate predictions.

Conclusion

The complexity of the problem was low, but it was having very high business impact. Many times, it's handy to use reverse engineering to solve the problem instead of jumping on new implementation or redesigning.

Please let me know your thoughts about the solution explained here or if you think the solution could be improved or if there is better alternative.

Feel free to reach out to me over LinkedIn if you have an interesting problem to solve!

Comments

40 · 4 comments

Like Comment Share

Add a comment...

Most recent

Bhawana Goel, PMP • 1st
PMP certified | Senior Scrum Master | Product Management | Business Analy...
3y

Interesting read Kunal with concise explanation

Like · 1 | Reply · 1 reply

Kunal Patil Author
Gen AI Solution Architect | Agentic AI • RAG • LangChain • AWS Bedro...
3y

Thanks Bhawana!

Like | Reply

Jitendra Tisge • 1st
Sr Scrum Master at Amadeus Labs | PMP® | SPC | SAFe 6 SM | ICP-ACC | SAF...
4y

Good stuff. I like the way you explained it so that anyone can easily understand and apply simple solution to problem. 👍 👍

Like · 1 | Reply · 1 reply

Kunal Patil Author 4y

Gen AI Solution Architect | Agentic AI • RAG • LangChain • AWS Bedro...

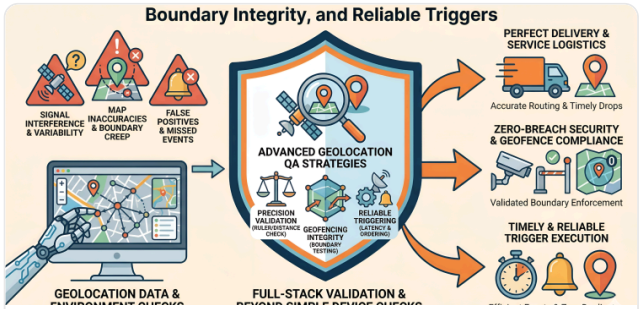
Thanks Jitendra!

Like | Reply

Kunal Patil

Gen AI Solution Architect | Agentic AI • RAG • LangChain • AWS Bedrock | Cloud-Native BFSI & Healthcare Leader | AWS Certified | 20+ Years experience

More articles for you



How Quality Engineering Ensures Precision and Reliable Location Triggers in Geolocation Systems?

Modern QA Insights

11 · 10 reposts

Revolutionizing Management Frameworks: An Exploration of Interface Identification in Policy Design

Walid A.

5

🚨 AIOps in Action: Turning Incidents into Intelligence

Jacob Sebastian

10 · 2 comments